Lab Course Automatic Control II

OPT I – Model predictive control of a laboratory crane

Supervisor: Paulina Conrad

1 Introduction

This experiment simulates the loading of a cargo via a gantry crane on the experimental setup shown in Figure 1. Gantry cranes are often used in cargo transfer areas and assembly halls to move loads horizontally and vertically at the same time. In this experiment, a *model predictive controller* (MPC) should be designed for this nonlinear, constrained multivariable system. Thereby, the limitations of the system shall be explicitly considered in the controller design. The designed model predictive controller will first be simulatively validated and then tested on the experimental setup, for which a real-time capable dSPACE system is available.

For the execution of this experiment, the contents of the lecture *Numerical Optimization and Model Predictive Control* are required. If necessary, re-read the most important topics with the help of [1].

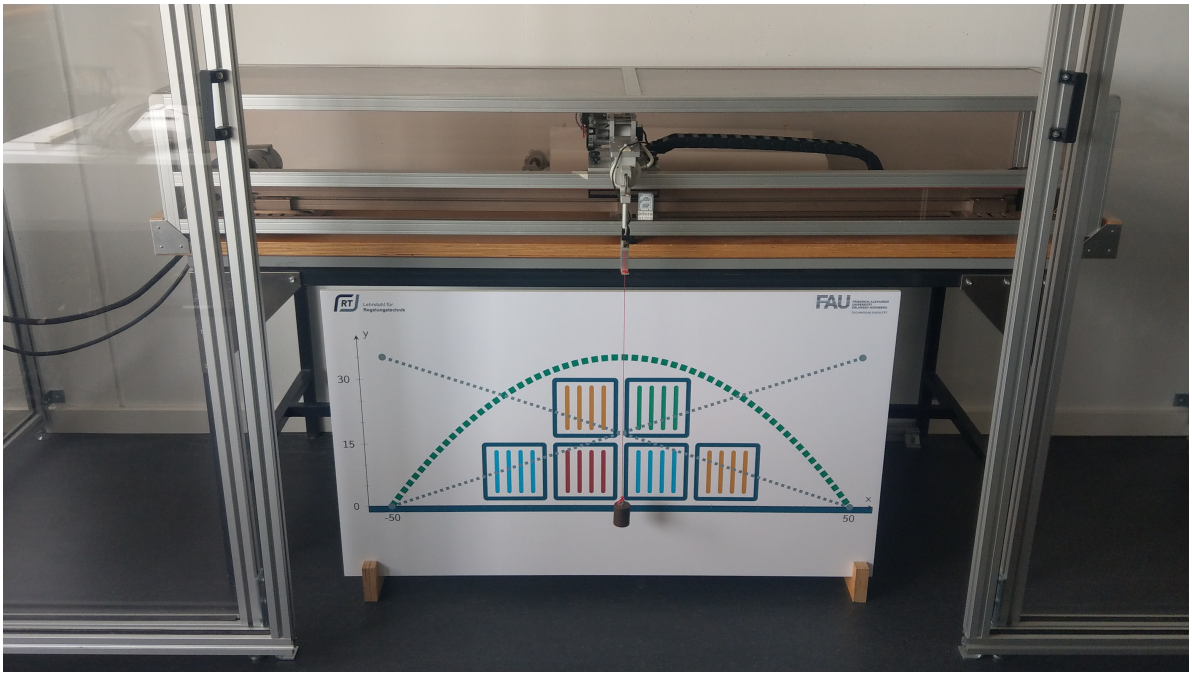

Figure 1: The laboratory crane available at the Chair of Automatic Control.

2 Experiment preparation

The basis for the design of a model predictive controller is always a model of the system to be controlled. Therefore, in a first step, a physical model of the laboratory crane is derived with the help of the *Lagrangian formalism*. The geometric relationships of the laboratory crane as well as the coordinate system chosen for modeling are shown schematically in Figure 2. The cart has the mass m_C and can be moved horizontally on a rail driven by the force F_C. In addition, the friction force F_F acts on the cart. On the cart is a winch motor with radius r_M, to which a load with mass m_L is attached via a rope. The rope with length ℓ is assumed to be massless and always taut. The winch has a moment of inertia J_M and can be driven by the torque M_M, allowing the rope to be retracted or extended. The following relationship applies

$$\dot{\ell} = \omega_M r_M \,, \tag{1}$$

where ω_M denotes the angular velocity of the winch. The winch is also subject to friction, which is described by the frictional torque M_F. The winch is supported so that the load can rotate freely about the z-axis via a pendulum bearing. In the following, the cart and the load are modeled as point masses.

In order to derive the equations of motion using the Lagrangian formalism, first the *Lagrangian function*

$$\mathcal{L} = \mathcal{T} - \mathcal{V} \tag{2}$$

is used, which is composed of the kinetic and potential energy $\mathcal{T}$ and $\mathcal{V}$ of the system. The energies $\mathcal{T}$ and $\mathcal{V}$ are described depending on so-called *generalized coordinates* $\boldsymbol{q}$, where for the laboratory crane $\boldsymbol{q} = [q_1, q_2, q_3]^\mathrm{T} = [x_C, \ell, \theta]^\mathrm{T}$ is chosen. The kinetic energy $\mathcal{T}$ is composed of the kinetic energy of the cart and the load as well as the rotational energy of the winch. The potential energy $\mathcal{V}$ of the system is equal to the potential energy of the load, since the potential energy of the cart becomes 0 due to the chosen coordinate system ($y_C = 0$). Thus, the equations of motion of the laboratory crane can be solved using the *Lagrangian equations of second kind*

$$\frac{\mathrm{d}}{\mathrm{d}t}\frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i \,, \quad i = 1, 2, 3 \,. \tag{3}$$

The vector $\boldsymbol{\tau} = [F_{x_C}, F_\ell, 0]^\mathrm{T}$ denotes the non-conservative forces in the direction of the generalized coordinates. In the case of the laboratory crane, these forces are composed of the forces generated by the electric motors on the cart and the winch, as well as the frictional forces.

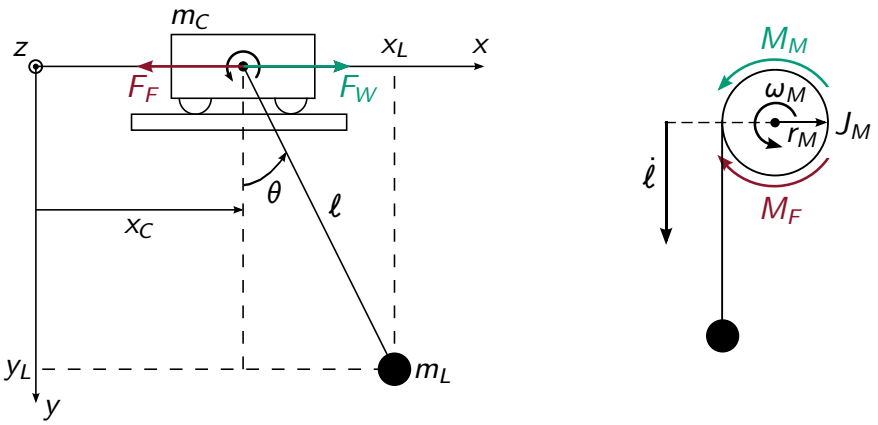

Figure 2: Geometric relations of the laboratory crane and the coordinate system chosen for modeling.

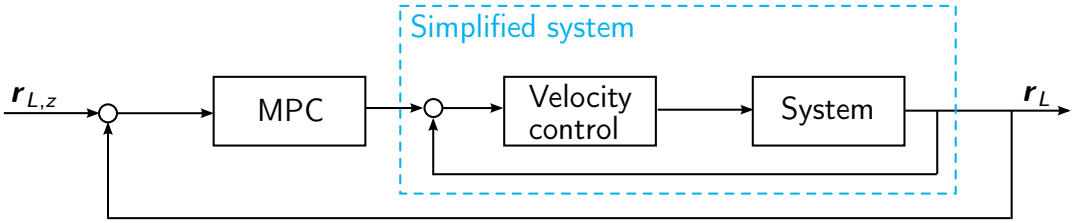

Figure 3: Cascaded control loop structure on real experimental setup.

(P1) *Calculate the equations of motion of the laboratory crane via* (3).

Note: *You can also perform the calculation using the Symbolic Math Toolbox from* MATLAB.

The structure of the control loop on the real experimental setup consists of the cascaded structure shown in Figure 3 with fast inner velocity controllers for the cart and rope velocities for friction compensation. Assuming that these inner velocity controllers are sufficiently fast and accurate, using the cart and rope acceleration a_x and a_ℓ, the dynamics can be simplified to

$$\ddot{x}_C = a_x \quad \ddot{\ell} = a_\ell\,, \quad \ddot{\theta} = \frac{1}{\ell}\left(-g\sin(\theta) - a_x\cos(\theta) - 2\dot{\ell}\dot{\theta}\right)\,. \tag{4}$$

In this lab experiment, a model predictive controller is developed based on the simplified model (4). To do this, the dynamics must be put into a state space representation of the form $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$.

(P2) *Bring the simplified dynamics* (4) *into the state space representation of the form* $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ *using the state and control vectors* $\boldsymbol{x} = [x_w, \dot{x}_w, \ell, \dot{\ell}, \theta, \dot{\theta}]^\mathsf{T}$ *and* $\boldsymbol{u} = [a_x, a_\ell]^\mathsf{T}$ *.*

State at least one alternative method to compensate for the frictional effects.

The aim of the lab is the movement of the load with the position

$$\boldsymbol{r}_L = \begin{bmatrix} x_L \\ \tilde{y}_L \end{bmatrix} = \begin{bmatrix} x_C + \sin(\theta)\ell \\ \ell_{y_0} - y_L \end{bmatrix} = \begin{bmatrix} x_C + \sin(\theta)\ell \\ \ell_{y_0} - \cos(\theta)\ell \end{bmatrix}\,, \tag{5}$$

from a starting position $\boldsymbol{r}_{L,0}$ to a target position $\boldsymbol{r}_{L,z}$. Here $\ell_{y_0} = 0.72\,\mathrm{m}$ denotes the rope length at which the lower edge of the load is located exactly at $y = 0$ in the coordinate system on the real experimental setup (see Figure 1).

(P3) *Formulate a suitable optimization problem for moving the load, taking into account the dynamics of the system, and state a corresponding quadratic cost functional for achieving the control objective. In addition to the deviation of the load from its target position, also penalize the states and control inputs of the system (with the exception of* x_C *and* ℓ*) by adapting the respective weights in order to be able to specifically influence the control result. Additionally take the limited performance of the drive motors into account, which are considered via the input constraints*

$$a_x \in \left[-9\,\frac{\mathrm{m}}{\mathrm{s}^2}, 9\,\frac{\mathrm{m}}{\mathrm{s}^2}\right]\,, \quad a_\ell \in \left[-3.5\,\frac{\mathrm{m}}{\mathrm{s}^2}, 3.5\,\frac{\mathrm{m}}{\mathrm{s}^2}\right]\,. \tag{6}$$

The numerical solution of the constrained nonlinear optimization problem is carried out within the scope of the lab using the MPC toolbox GRAMPC. It is based on an *augmented Lagrangian method* for the direct consideration of state-dependent constraints in combination with a underlying *gradient method*, which is based on the necessary *first-order optimality conditions*.

(P4) *Formulate the first-order optimality conditions for your optimization problem from task **(P3)** and outline the procedure of the numerical solution using the gradient method.*

Furthermore, the physical limitations of the experimental setup should be directly considered in the control. The operating space is limited by the constraints

$$x_C \in [x_{W,\min}, x_{W,\max}] = [-0.6\,\mathrm{m}, 0.6\,\mathrm{m}]\,, \quad \ell \in [\ell_{\min}, \ell_{\max}] = [0.25\,\mathrm{m}, 0.8\,\mathrm{m}]\,. \tag{7}$$

(P5) *Extend your optimization problem from task **(P3)** to include the operating space constraints of the laboratory crane. How are such state-dependent constraints taken into account during numerical solution with the MPC toolbox GRAMPC?*

Now the designed model predictive controller will be realized and simulatively validated using the MPC toolbox GRAMPC.

(P6) *Install the MPC toolbox GRAMPC and familiarize yourself with its use by reading the documentation [3] and the articles [4] and [5].*

(P7) *Realize the designed model predictive controller using the MPC toolbox GRAMPC. To do so, implement the system dynamics, the cost function and the constraints as well as the partial derivatives needed for the numerical solution in a `probfct.c` file. Then create a corresponding initialization script `initData.m` and pass the weights of your cost function via the `userparam` vector.*

***Note:** A corresponding template can be found in `OPT1_Template/V7_Template`. General templates are available in GRAMPC under `examples/Template`. For the calculation of the partial derivatives you can use the Symbolic Math Toolbox from MATLAB.*

(P8) *Test the controller simulatively for the position changes of the load from*

$$\boldsymbol{r}_{L,0} = \begin{bmatrix} -0.55\,\mathrm{m} \\ 0.0\,\mathrm{m} \end{bmatrix} \tag{8}$$

to the target postion

$$\boldsymbol{r}_{L,z_1} = \begin{bmatrix} 0.55\,\mathrm{m} \\ 0.0\,\mathrm{m} \end{bmatrix} \quad \textit{and} \quad \boldsymbol{r}_{L,z_2} = \begin{bmatrix} 0.55\,\mathrm{m} \\ 0.35\,\mathrm{m} \end{bmatrix}\,, \tag{9}$$

*by constructing the control loop under Simulink. Use the Simulink template under `OPT1_Template/V7_Template`. Use the simplified dynamics of the laboratory crane derived in task **(P2)** as the system model, as well as a prediction horizon $T_h = 1\,\mathrm{s}$ and a sampling rate of $\Delta t = 1\,\mathrm{ms}$.*

(P9) *Now vary the prediction horizon T_h, the sampling time Δt, the number of gradient iterations per step (option `MaxGradIter`) as well as the weights in the cost function and in each case describe the influence of an increase / decrease on the control result.*

Note: The goal of the experiment execution is to test your model predictive control developed in the preparation on the real system. Therefore, be sure to bring all relevant files (Simulink model, GRAMPC probfct, initialization script, etc.) from the preparation to the execution of the experiment. Since MATLAB R2018b is installed on the lab computer, save your Simulink model in this version if you are using a newer MATLAB version to avoid complications (with `File` → `Export Model to` → `Previous Model...`).

3 Experiment execution

During the execution of the experiment, the controller designed and simulatively validated in the preparation is tested on the real system. Before starting the experiment, read the general instructions on carrying out the experiment in Section 3.1 carefully.

3.1 General instructions for experiment execution

- Save at least one measurement for each task of the experiment execution. This is done by right clicking in the plot window in ControlDesk and then "Save Displayed Data as New Measurement". Next, the measurement data, which is now available in `Project` → `Praktikum_RT2_MPC_Verladebruecke` → `Measurements`, can be saved as a MATLAB file and loaded and visualized using MATLAB.
- For the execution of the experiment, the doors of the experimental setup must always be closed, otherwise a safety switch will no longer be activated and the laboratory crane can no longer be operated.
- When carrying out the experiment, pay particular attention to the instructions in the individual tasks.

3.2 Execution

Copy your initialization script and your `probfct.c` file from the preparation to the `C:\Verladebruecke_MPC\code` directory. Then rename the probfct file to "probfct_Verladebruecke.c" to ensure that the compiler finds it later.

Next, open MATLAB on the lab computer. If a prompt concerning the dSPACE system appears, select "RTI1104". Acknowledge all further messages with `OK`. Change to the `C:\Verladebruecke_MPC\code` directory and open the Simulink model `Verladebruecke_MPC.slx`. In this model, a measurement filtering as well as a inner velocity control for friction compensation have already been implemented.

(E1) *Integrate your model predictive controller implemented in the preparation into the existing Simulink template for controlling the laboratory crane under `Regelgruppe` → `MPC-basierte Regelung`. The available inputs are the current measured system states $\boldsymbol{x}_k$ and the current time t_k. Link the outputs with the outputs of your controller as well as the current target position of the load.*

To ensure real-time capability of the control on the dSPACE system at the real system, for all following tasks select the prediction horizon (parameter `Thor`) $T_h = 1\,\mathrm{s}$, a MPC sampling time (parameter `dt`) of 10 ms, 6 gradient and 2 multiplier iterations (options `MaxGradIter` & `MaxMultIter`), 15 grid points for numerical integration (option `Nhor`), and the Euler integration scheme (option `Integrator`).

(E2) *Adjust the options and parameters in your initialization script accordingly.*

To use the designed controller on the real system, the Simulink model must be compiled and loaded onto the dSPACE system. This can be done either by the *Build-Model* icon in Simulink (rightmost in the top toolbar) or by the key shortcut *Ctrl-B*. Additionally, this will create a *variable description* file (`.sdf`) in the current directory, which will be needed later.

(E3) *Execute your own initialization script as well as the already existing script* `script_dSpace.m`*, which contains the parameters for the inner velocity control as well as the measurement filtering. Next, compile the Simulink model and fix any error messages that may appear in the process.*

In the next step, start the *dSPACE ControlDesk* program and open the prepared project via `File` → `Open` → `Open Project and Experiment` → `Root Directory` → `Symbol ''new folder''` → select `C:\Verladebruecke_MPC\Praktikum_RT2_MPC_Verladebruecke` → `Versuchsdurchfuehrung` → `OK`.

The project provides you with a graphical user interface for controlling the laboratory crane and displaying the target and actual position of the load as well as the control inputs. With the `Go Online` and `Start Measuring` icons in the upper toolbar you start the online control and the visualization of the measured values. Before compiling a new version of your Simulink model, you must click `Go Offline`, otherwise an error message will appear during the building process. After **each** compile, you must build the new version of the `.sdf` file in ControlDesk using `Project` → `Praktikum_RT2_MPC_Verladebruecke\Hardware Configurations\Platform\verlade-bruecke_mpc.sdf` → `Right click` → `Reload` → `Yes`.

To enable efficient parameterization of your model predictive controller in online mode on the real experimental setup, it makes sense to introduce the most important parameters (weights in the cost function and the specified target point $\boldsymbol{r}_{L,z}$) as variable parameters that can be adjusted via the ControlDesk interface. This enables parameterization in online mode without having to recompile the Simulink model each time.

(E4) *Define the weights of your cost function and the parameters of the target trajectories as* MATLAB *variables in an initialization script. Then insert the corresponding parameters into the* `userparam` *and* `xdes` *vectors in the Simulink model before passing them to the GRAMPC block.*

To be able to adjust the parameters in the ControlDesk interface, insert an input field for each variable via `Layouting` → `Insert Instrument` → `Numeric Input`*. Then you can link the respective variable to the input field by* `Right click on input field` → `Variables` → `Add`*.*

Note: *You must first recompile your Simulink model after the changes and reload the* `.sdf` *file, otherwise the parameters will not yet be available in ControlDesk.*

(E5) *Switch on the control hardware of the laboratory crane (gray box on the right of the test setup). Then switch to online mode in the ControlDesk interface. Enable the motor and perform a calibration. This is always necessary when either a new* `.sdf` *file has been loaded or the laboratory crane has reached one of its system limits. You can then move to a choosable starting point.*

(E6) *Next, test your controller for the same position changes as in preparation task* ***(P8)****. To do this, move to the starting point using "Move to starting point" and then start your controller by selecting "MPC-based control".*

If your controller works in principle, adjust the weights in the cost function so that the target position is reached as fast as possible, but without extreme overshoot. What is the approximate minimum time required by the laboratory crane for the two position changes? How do the trajectories differ at the operating point change to $\boldsymbol{r}_{L,z_1}$ *for a high and low weighting of* $\Delta\tilde{y}_L$*, respectively?*

Now an automatic sequence of operating point changes is realized using the optimal parameterization found in task **(E6)**. The relevant operating points are

$$r_{L,0} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}, \quad r_{L,1} = \begin{bmatrix} -0.5 \\ 0.0 \end{bmatrix}, \quad r_{L,2} = \begin{bmatrix} -0.5 \\ 0.35 \end{bmatrix}, \quad r_{L,3} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad r_{L,4} = \begin{bmatrix} 0.5 \\ 0.35 \end{bmatrix} \tag{10}$$

(all values in m), which are to be approached in the following order

$$r_{L,0} \to r_{L,1} \to r_{L,2} \to r_{L,3} \to r_{L,4} \to r_{L,1} \to r_{L,3} \to r_{L,0}\,. \tag{11}$$

(E7) *Realize the automatic traversal of the sequence of setpoints by adjusting the target positions of the load depending on the current time t_k in your Simulink model. Make sure that the time for the individual operating point changes is at least as long as the required time determined in task **(E6)**.*

***Note:** For this purpose, you can adapt the `xdes` vector depending on the time t_k, for example, using a `Matlab Function` block.*

Another important task of laboratory cranes is to move a load over an obstacle, e.g. a stack of containers. The height of the obstacle h_c shall be defined in the following as a parabola of the form

$$h_c(x_L) = p_2 x_L^2 + p_1 x_L + p_0 \tag{12}$$

dependent on the x-position of the load x_L as well as the parameters p_0, p_1 and p_2, which can be derived from the constraints

$$h_c(-0.5) = 0.0\,, \quad h_c(0.0) = 0.35\,, \quad h_c(0.5) = 0.0\,. \tag{13}$$

Since the load is not a point mass but an extended object, the constraint should be satisfied for the center of the lower edge of the load as well as for the lower left and right corners, i.e.

$$\tilde{y}_L \geq h_c(x_L) \quad \& \quad \tilde{y}_L \geq h_c(x_L + b_L/2) \quad \& \quad \tilde{y}_L \geq h_c(x_L - b_L/2) \tag{14}$$

be satisfied, where $b_L = 3.5\,\text{cm}$ describes the width of the load.

(E8) *Compute the parameters p_0, p_1 and p_2 and extend your problem formulation by the constraints (14) by first bringing them to the required form $\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{u}) \leq \boldsymbol{0}$ and then integrating them into your problem formulation. Additionally, add a constraint on the cart speed of $|\dot{x}_C| \leq 0.1\,\text{m}\,\text{s}^{-1}$ to better observe the transfer of the load over the obstacle.*

*Next, test your extensions for an operating point change from $\boldsymbol{r}_{L,0} = [-0.55, 0]^\mathsf{T}$ m to $\boldsymbol{r}_{L,z} = [0.55, 0]^\mathsf{T}$ m and adjust the weights in the cost function to achieve a good control result. To do this, undo your changes from task **(E7)** to be able to specify a single target position again via the ControlDesk interface. Is the constraint exactly met?*

Finally, the behavior of the control is investigated in the event of a manual fault being applied. For this purpose, the doors of the experimental setup must be opened, which means that a safety switch is no longer actuated and the laboratory crane can no longer be operated. Therefore, contact the supervisor to switch off this safety feature.

(E9) *Move the load to a position in the center of the workspace and test how the control reacts to manually disturbing the load from this position. How does the reaction differ with high / low weighting of the rope angle?*

References

[1] K. Graichen. Numerical Optimization and Model Predictive Control. *Lecture notes Friedrich-Alexander Universität Erlangen-Nürnberg*, 2023.

[2] B. Käpernick. Gradient-based nonlinear model predictive control with constraint transformation for fast dynamical systems. Shaker Verlag, Aachen, 2016.

[3] T. Englert, A. Völz, F. Mesmer, S. Rhein, K. Graichen. GRAMPC documentation. `sourceforge.net/projects/grampc`, 2018.

[4] B. Käpernick, K. Graichen. The gradient based nonlinear model predictive control software GRAMPC. In *Proc. European Control Conference (ECC)*, Seiten 1170-1175, Straßburg, Frankreich, 2014.

[5] T. Englert, A. Völz, F. Mesmer, S. Rhein, K. Graichen. A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach. *Optimization and Engineering*, `doi.org/10.1007/s11081-018-9417-2`, 2019.